📋 Worksheet 3 – Cost Optimization Debate

Nhóm: EcoSense Team **Thời gian:** 10:15 – 10:50

---

3 Chiến lược optimization phù hợp nhất với EcoSense

✅ Chiến lược 1: Prompt Compression

Tiết kiệm phần nào	Token input – hiện đang ~850 token/request, phần lớn là system prompt và JSON data từ API
Lợi ích	Giảm trực tiếp chi phí LLM, tác động ngay lập tức
Trade-off	Nếu nén quá mạnh → mất context → response kém chất lượng, gợi ý sai
Thời điểm áp dụng	**Làm ngay** – không cần thay đổi kiến trúc, chỉ cần tối ưu lại system prompt và cách format JSON API response trước khi inject vào

---

✅ Chiến lược 2: Semantic Caching

Tiết kiệm phần nào	Giảm số lần gọi LLM – các câu hỏi tương tự nhau ("Hà Nội có mưa không?", "Hôm nay Hà Nội mưa không?") sẽ dùng lại response đã cache
Lợi ích	Rất hiệu quả với EcoSense vì câu hỏi về thời tiết/AQI có tính lặp lại cao, đặc biệt trong peak sáng
Trade-off	Cần thêm vector database để so sánh semantic similarity → tăng độ phức tạp hệ thống; cache stale nếu dữ liệu thời tiết thay đổi nhanh
Thời điểm áp dụng	**Để sau** – áp dụng khi traffic đủ lớn để cache có hiệu quả, MVP chưa cần thiết

---

✅ Chiến lược 3: Selective Inference – Phân tầng request

Tiết kiệm phần	Không phải câu hỏi nào cũng cần gọi LLM – các câu hỏi đơn giản có thể trả lời bằng

nào	rule-based hoặc template
Lợi ích	Giảm đáng kể số lần gọi LLM, ví dụ: *"AQI Hà Nội hôm nay?"* → trả về data thẳng từ API không cần LLM suy luận
Trade-off	Cần thiết kế thêm tầng phân loại intent; nếu phân loại sai → trả lời cứng nhắc, mất trải nghiệm chat tự nhiên
Thời điểm áp dụng	**Để sau** – cần đủ data về pattern câu hỏi thực tế của người dùng mới thiết kế được tầng phân loại chính xác

---

🗓️ Chốt thứ tự thực hiện

Thời điểm	Chiến lược	Lý do
Làm ngay	Prompt Compression	Đơn giản, tác động lớn nhất, không cần thay đổi kiến trúc
Khi scale ~5x	Semantic Caching	Lúc này traffic đủ lớn để cache sinh lời
Khi có production data	Selective Inference	Cần biết người dùng hỏi gì nhiều nhất mới thiết kế được

---

Đầu ra: ✅ Worksheet 3 hoàn thành